

Dark Factory · pocketful track

A three-seat headless factory on band-sdk (Claude Code)

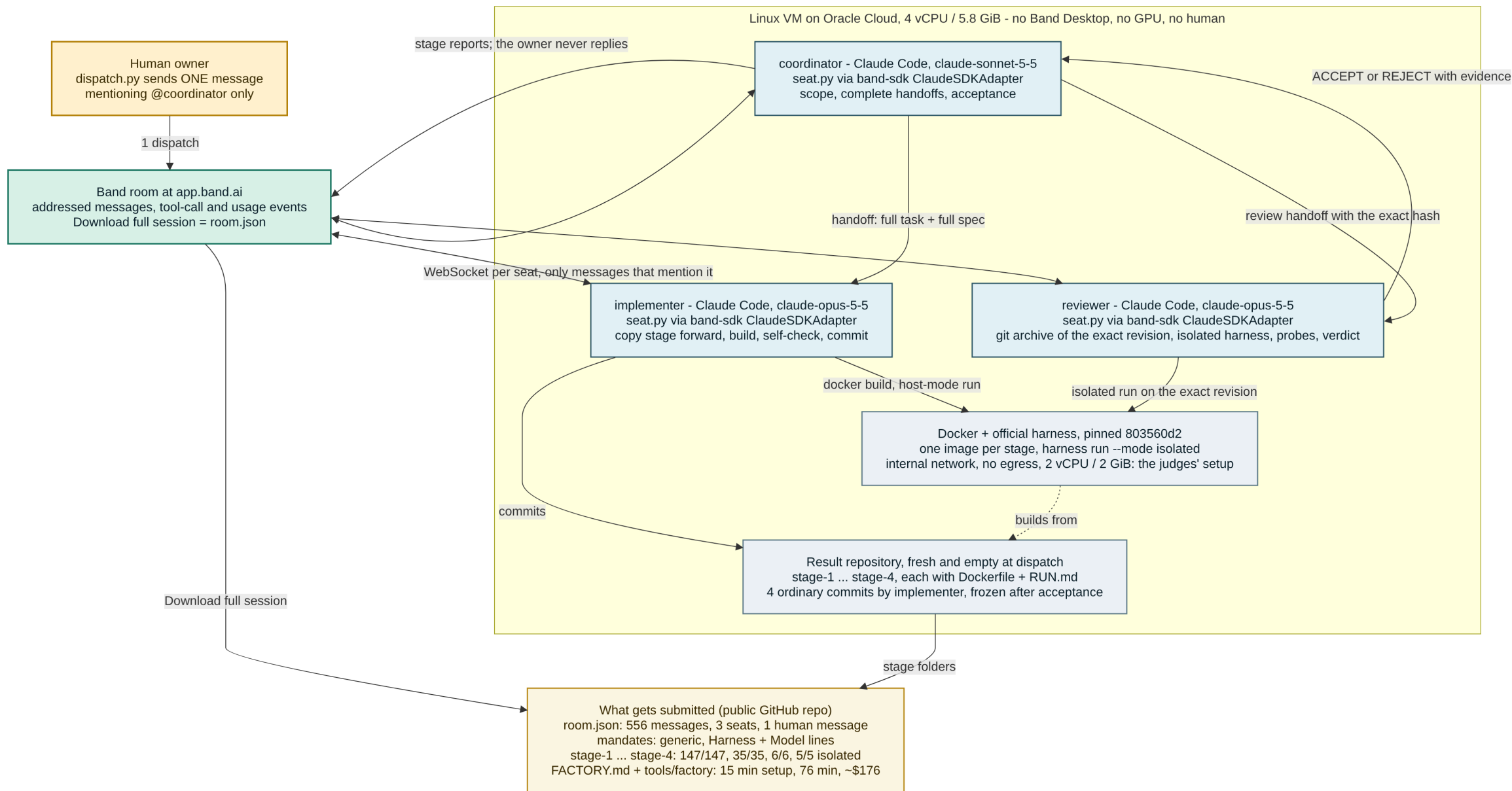
One human dispatch. Zero steering. Four stages accepted in 76 minutes.

Felipe Salvego (Jazz Automations) & Carlos Henrique

github.com/jazzautomations/pocketful-dark-factory

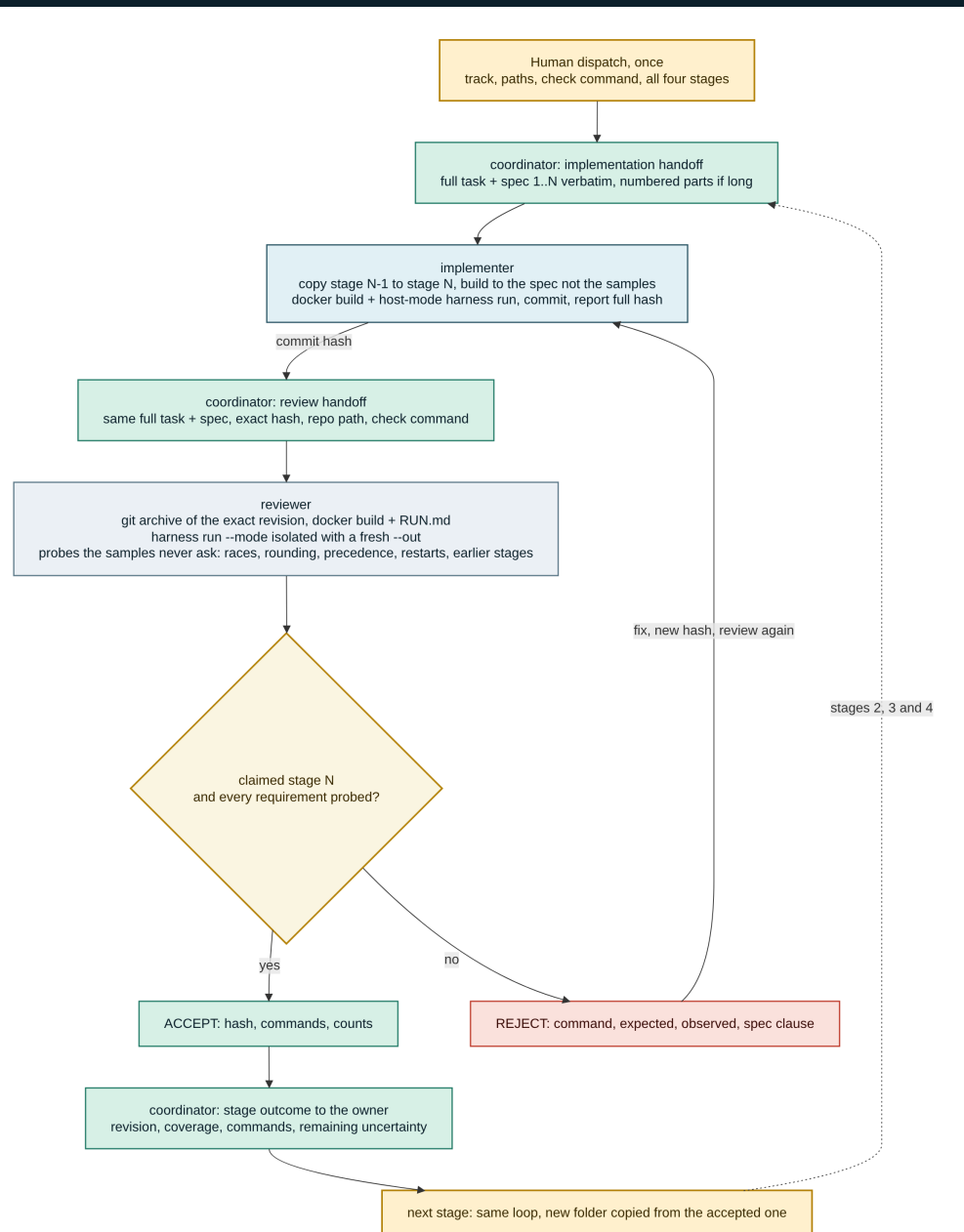
The factory, in one picture

- **coordinator** (claude-sonnet-5-5): scope, sequencing, complete handoffs, acceptance, reporting
- **implementer** (claude-opus-5-5): one scoped item at a time, builds, self-checks, commits, reports the hash
- **reviewer** (claude-opus-5-5): reproduces on the exact revision, runs the official harness in isolated mode, probes beyond the shipped checks
- Three Band **external agents**, each a Python process: band-sdk keeps the room socket, every addressed message becomes one Claude Code turn with the seat's mandate
- No Band Desktop, no GUI, no human after the dispatch
- Mandates are generic: swap the dispatch and the same band builds something else



Why headless, and what it cost us

- Band Desktop would not run on any machine we had. We read the band-sdk ClaudeSDKAdapter source and wrote **seat.py** (60 lines): mandate as system section, permission mode **auto** (not bypass), cwd pinned to the result repo, per-seat git identity
- Seats only receive messages that mention them, so **every handoff carries the whole task and the whole spec**, split into numbered parts (stage 2: 4 parts). Large prompts, but nobody ever works from a summary
- Rehearsed on the organiser's **toy** track first: one dispatch, stage 1 accepted in 2 minutes, harness check gates 1, 2 and 4 green
- Setup from a bare Ubuntu VM to a running band: about 15 minutes, all in FACTORY.md



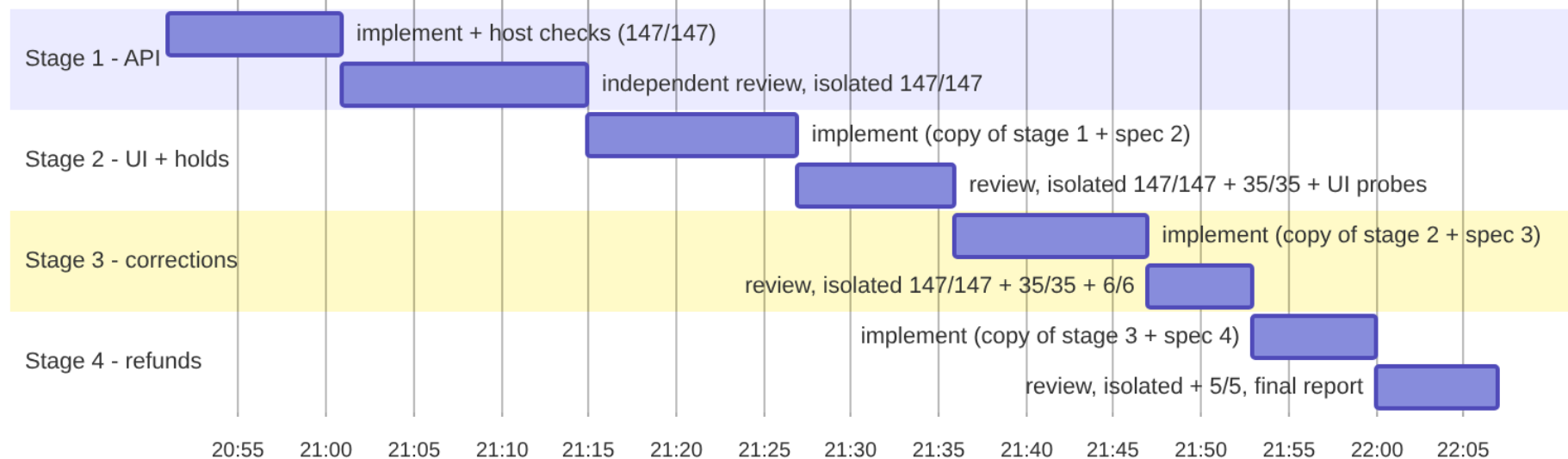
The submitted run (room.json, 556 messages, 1 human message)

Stage	Accepted	Reviewer's isolated harness run	Reviewer's own probes
1 API	+24 min	147/147	50-way idempotency races, rounding, error precedence, export/import
2 UI + holds	+45 min	147/147 · 35/35	real Chromium: pay-form idempotency, refresh wins, 375 px, upgrade from stage 1 export
3 time & corrections	+62 min	147/147 · 35/35 · 6/6	as_of/known_at, snapshots, 20-way same-revision race = 1 winner
4 refunds & batches	+76 min	147/147 · 35/35 · 6/6 · 5/5	real imports of stage 1–3 exports, settlement membership preserved

Reviewer rejections: **0**. Human messages after dispatch: **0**. Weak spots were still reported (a `limit=4%0A` quirk found at stage 1, fixed in the stage 2 copy).

`tools/factory/usage.py` recomputes every number from `room.json`

Submitted run - 2026-10-04 - dispatch 20:51 UTC, final report 22:07 UTC (76 min, 0 rejections, 0 human messages)



How bad work gets caught

- ACCEPT must name the revision hash, the exact harness command and the counts; a pass without evidence is a REJECT by mandate
- REJECT carries command, expected, observed and the spec clause; the implementer fixes exactly that and reports a new hash; the reviewer checks the new hash, never the working tree
- Accepted stage folders are frozen; the reviewer checks they stay byte-identical, and that no nested .git, submodule, symlink or credential entered the repo
- The coordinator posts each stage outcome **with remaining uncertainty**, so weak spots are on the record even on first-pass acceptance

What it built

- Standard-library Python service, one image per stage, pinned multi-arch base, no network at run time
- Wallet, payments, requests, splits with exact share arithmetic, feed, atomic settlements
- Browser UI: send, request, split, holds and captures; survives lost responses and stale state
- Revision ledger with as_of / known_at, frozen statements, corrections, refunds, operator batches

Measured spend

76 min

wall clock, dispatch to final report

~\$176

at list price (adapter estimate): coordinator
\$14 · implementer \$100 · reviewer \$63

406k output tokens · 791k cache writes

Reuse it

- **mandates/**: three generic seats, nothing about wallets in them
- **tools/factory/**: seat.py, start.sh, dispatch.py, usage.py
- **docs/dispatch.txt**: the single human input, verbatim
- **room.json**: the full session download, unchanged
- **docs/video-factory-run.mp4**: the room working and the app it produced

github.com/jazzautomations/pocketful-dark-factory